FASTAPI: COMPLETE REFERENCE GUIDE (Part 1)

Table of Contents

1. [Python Types Intro](#1-python-types-intro)
2. [Concurrency and Async/Await](#2-concurrency-and-asyncawait)
3. [First Steps](#3-first-steps)
4. [Path Parameters](#4-path-parameters)
5. [Query Parameters](#5-query-parameters)
6. [Request Body](#6-request-body)
7. [Query Parameters and String Validations](#7-query-parameters-and-string-validations)
8. [Path Parameters and Numeric Validations](#8-path-parameters-and-numeric-validations)
9. [Query Parameter Models](#9-query-parameter-models)
10. [Body - Multiple Parameters](#10-body---multiple-parameters)
11. [Body - Fields](#11-body---fields)
12. [Body - Nested Models](#12-body---nested-models)
13. [Declare Request Example Data](#13-declare-request-example-data)
14. [Extra Data Types](#14-extra-data-types)
15. [Cookie Parameters](#15-cookie-parameters)
16. [Header Parameters](#16-header-parameters)
17. [Cookie Parameter Models](#17-cookie-parameter-models)
18. [Header Parameter Models](#18-header-parameter-models)
19. [Response Model - Return Type](#19-response-model---return-type)
20. [Extra Models](#20-extra-models)
21. [Response Status Code](#21-response-status-code)
22. [Form Data](#22-form-data)
23. [Form Models](#23-form-models)
24. [Request Files](#24-request-files)
25. [Request Forms and Files](#25-request-forms-and-files)
26. [Handling Errors](#26-handling-errors)
27. [Path Operation Configuration](#27-path-operation-configuration)
28. [JSON Compatible Encoder](#28-json-compatible-encoder)
29. [Body - Updates](#29-body---updates)
30. [Dependencies](#30-dependencies)
31. [Classes as Dependencies](#31-classes-as-dependencies)
32. [Sub-dependencies](#32-sub-dependencies)
33. [Dependencies in Path Operation Decorators](#33-dependencies-in-path-operation-decorators)
34. [Global Dependencies](#34-global-dependencies)
35. [Dependencies with Yield](#35-dependencies-with-yield)
36. [Security - First Steps](#36-security---first-steps)
37. [Get Current User](#37-get-current-user)
38. [Simple OAuth2 with Password and Bearer](#38-simple-oauth2-with-password-and-bearer)
39. [OAuth2 with Password and JWT Tokens](#39-oauth2-with-password-and-jwt-tokens)
40. [Middleware](#40-middleware)
41. [CORS](#41-cors)
42. [SQL (Relational) Databases](#42-sql-relational-databases)
43. [Bigger Applications - Multiple Files](#43-bigger-applications---multiple-files)

44. [Stream JSON Lines](#44-stream-json-lines)
45. [Server-Sent Events (SSE)](#45-server-sent-events-sse)
46. [Background Tasks](#46-background-tasks)
47. [Metadata and Docs URLs](#47-metadata-and-docs-urls)
48. [Frontend](#48-frontend)
49. [Static Files](#49-static-files)
50. [Testing](#50-testing)
51. [Debugging](#51-debugging)

---

1. Python Types Intro

Python has support for optional "type hints" (also called "type annotations") that allow declaring the type of variables.

Motivation

Type hints provide:
- Better editor support (autocompletion)
- Error checks
- Better documentation

Basic Type Annotations

```python
def get_full_name(first_name: str, last_name: str) -> str:
    full_name = first_name.title() + " " + last_name.title()
    return full_name
```

Simple Types

```python
def process_item(item_a: str, item_b: int, item_c: float, item_d: bool, item_e: bytes):
    return item_a, item_b, item_c, item_d, item_e
```

Generic Types (List, Tuple, Set, Dict)

```python
# List of strings
def process_items(items: list[str]):
    for item in items:
        print(item)

# Tuple
def process_items(items_t: tuple[int, int, str], items_s: set[bytes]):
    return items_t, items_s
```

```python
# Dict
def process_items(prices: dict[str, float]):
    for item_name, item_price in prices.items():
        print(item_name, item_price)
```

Union Types

```python
def process_item(item: int | str):
    print(item)

# Optional values
def say_hi(name: str | None = None):
    if name is not None:
        print(f"Hey {name}!")
```

Classes as Types

```python
class Person:
    def __init__(self, name: str):
        self.name = name

def get_person_name(one_person: Person):
    return one_person.name
```

Pydantic Models

```python
from pydantic import BaseModel

class User(BaseModel):
    id: int
    name: str = "John Doe"
    signup_ts: datetime | None = None
    friends: list[int] = []

user = User(id=123, name="Jane")
```

Annotated Types

```python
from typing import Annotated
```

```
def say_hello(name: Annotated[str, "this is just metadata"]) -> str:
    return f"Hello {name}"
```

---

2. Concurrency and Async/Await

Quick Summary

- Use `async def` with `await` for I/O-bound operations
- Use normal `def` for CPU-bound operations or when using synchronous libraries
- FastAPI can mix both types

Async Example

```python
@app.get('/')
async def read_results():
    results = await some_library()
    return results
```

Sync Example

```python
@app.get('/')
def results():
    results = some_library()
    return results
```

Concurrency vs Parallelism

Concurrency: Multiple tasks making progress at the same time (like waiting for burgers while talking to someone)

Parallelism: Multiple tasks actually executing simultaneously (multiple processors doing work)

async/await Syntax

```python
# Coroutine function
async def get_burgers(number: int):
    # Do some async work
    return burgers

# Calling a coroutine
burgers = await get_burgers(2)
```

```

### Path Operation Functions

- `async def`: Run in the main event loop (good for I/O)
- `def`: Run in an external threadpool (good for CPU-bound work)

---

## 3. First Steps

### Install FastAPI

```bash
uv init awesome-project --bare
cd awesome-project
uv add "fastapi[standard]"
```

Basic App

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Hello World"}
```

Run the App

```bash
uv run fastapi dev
```

Interactive API Docs

- http://127.0.0.1:8000/docs (Swagger UI)
- http://127.0.0.1:8000/redoc (ReDoc)

---

4. Path Parameters

Basic Path Parameter

```python

```python
@app.get("/items/{item_id}")
async def read_item(item_id: int):
    return {"item_id": item_id}
```

Path Parameters with Types

```python
@app.get("/items/{item_id}")
async def read_item(item_id: int):
    return {"item_id": item_id}
```

Data Validation

FastAPI automatically validates path parameters and converts them to the specified type.

Order Matters

Path operations are evaluated in order:

```python
@app.get("/users/me")
async def read_user_me():
    return {"user_id": "the current user"}

@app.get("/users/{user_id}")
async def read_user(user_id: str):
    return {"user_id": user_id}
```

Predefined Values

```python
from enum import Enum

class ModelName(str, Enum):
    alexnet = "alexnet"
    resnet = "resnet"
    lenet = "lenet"

@app.get("/models/{model_name}")
async def get_model(model_name: ModelName):
    if model_name is ModelName.alexnet:
        return {"model_name": model_name, "message": "Deep Learning FTW!"}
    return {"model_name": model_name}
```

Path Converters

```python
@app.get("/files/{file_path:path}")
async def read_file(file_path: str):
    return {"file_path": file_path}
```

---

5. Query Parameters

Basic Query Parameters

```python
@app.get("/items/")
async def read_item(skip: int = 0, limit: int = 10):
    return fake_items_db[skip : skip + limit]
```

Optional Query Parameters

```python
@app.get("/items/{item_id}")
async def read_item(item_id: str, q: str | None = None):
    if q:
        return {"item_id": item_id, "q": q}
    return {"item_id": item_id}
```

Type Conversion

```python
@app.get("/items/{item_id}")
async def read_item(item_id: str, q: str | None = None, short: bool = False):
    item = {"item_id": item_id}
    if not short:
        item.update({"description": "This is an amazing item"})
    return item
```

Required Query Parameters

```python
@app.get("/items/{item_id}")
async def read_user_item(item_id: str, needy: str):
    item = {"item_id": item_id, "needy": needy}
    return item
```

Multiple Parameters

```python
@app.get("/users/{user_id}/items/{item_id}")
async def read_user_item(
    user_id: int, item_id: str, q: str | None = None, short: bool = False
):
    item = {"item_id": item_id, "owner_id": user_id}
    if q:
        item.update({"q": q})
    return item
```

---

6. Request Body

Using Pydantic Models

```python
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None

@app.post("/items/")
async def create_item(item: Item):
    return item
```

Body + Path Parameters

```python
@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
    return {"item_id": item_id, **item.model_dump()}
```

Body + Path + Query Parameters

```python
@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item, q: str | None = None):
    result = {"item_id": item_id, **item.model_dump()}
    if q:
        result.update({"q": q})
```

```
    return result
```

Model Methods

```python
# Convert model to dict
item_dict = item.model_dump()

# Access attributes
item.name
item.price
```

---

7. Query Parameters and String Validations

Using Query with Annotated

```python
from typing import Annotated
from fastapi import Query

@app.get("/items/")
async def read_items(q: Annotated[str | None, Query(max_length=50)] = None):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

String Validations

```python
@app.get("/items/")
async def read_items(
    q: Annotated[str | None, Query(min_length=3, max_length=50, pattern="^fixedquery$")] = None,
):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

Default Values

```python
@app.get("/items/")
```

```
async def read_items(q: Annotated[str, Query(min_length=3)] = "fixedquery"):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

Required with Query

```python
@app.get("/items/")
async def read_items(q: Annotated[str, Query(min_length=3)]):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

Multiple Values

```python
@app.get("/items/")
async def read_items(q: Annotated[list[str] | None, Query()] = None):
    query_items = {"q": q}
    return query_items
```

Metadata

```python
@app.get("/items/")
async def read_items(
    q: Annotated[
        str | None,
        Query(
            title="Query string",
            description="Query string for the items to search",
            min_length=3,
            deprecated=True,
        )
    ] = None,
):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

Alias Parameters

```python
@app.get("/items/")
async def read_items(q: Annotated[str | None, Query(alias="item-query")] = None):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results
```

Exclude from OpenAPI

```python
@app.get("/items/")
async def read_items(
    hidden_query: Annotated[str | None, Query(include_in_schema=False)] = None,
):
    if hidden_query:
        return {"hidden_query": hidden_query}
    return {"hidden_query": "Not found"}
```

Custom Validation with AfterValidator

```python
from pydantic import AfterValidator

def check_valid_id(id: str):
    if not id.startswith(("isbn-", "imdb-")):
        raise ValueError('Invalid ID format')
    return id

@app.get("/items/")
async def read_items(
    id: Annotated[str | None, AfterValidator(check_valid_id)] = None,
):
    return {"id": id}
```

---

8. Path Parameters and Numeric Validations

Using Path with Annotated

```python
from fastapi import Path

@app.get("/items/{item_id}")
```

```
async def read_items(
    item_id: Annotated[int, Path(title="The ID of the item to get")],
    q: Annotated[str | None, Query(alias="item-query")] = None,
):
    results = {"item_id": item_id}
    if q:
        results.update({"q": q})
    return results
```

Number Validations

```python
@app.get("/items/{item_id}")
async def read_items(
    item_id: Annotated[int, Path(title="The ID of the item to get", ge=1)],
    q: str,
):
    results = {"item_id": item_id}
    if q:
        results.update({"q": q})
    return results

@app.get("/items/{item_id}")
async def read_items(
    item_id: Annotated[int, Path(gt=0, le=1000)],
    q: str,
):
    results = {"item_id": item_id}
    if q:
        results.update({"q": q})
    return results
```

Float Validations

```python
@app.get("/items/{item_id}")
async def read_items(
    *,
    item_id: Annotated[int, Path(ge=0, le=1000)],
    q: str,
    size: Annotated[float, Query(gt=0, lt=10.5)],
):
    results = {"item_id": item_id}
    if q:
        results.update({"q": q})
    if size:
        results.update({"size": size})
```

```
    return results
```

Validation Parameters Summary

- `gt`: greater than
- `ge`: greater than or equal
- `lt`: less than
- `le`: less than or equal

---

9. Query Parameter Models

Using Pydantic Models for Query Parameters

```python
from pydantic import BaseModel, Field

class FilterParams(BaseModel):
    limit: int = Field(100, gt=0, le=100)
    offset: int = Field(0, ge=0)
    order_by: Literal["created_at", "updated_at"] = "created_at"
    tags: list[str] = []

@app.get("/items/")
async def read_items(filter_query: Annotated[FilterParams, Query()]):
    return filter_query
```

Forbid Extra Parameters

```python
class FilterParams(BaseModel):
    model_config = {"extra": "forbid"}
    limit: int = Field(100, gt=0, le=100)
    offset: int = Field(0, ge=0)
    order_by: Literal["created_at", "updated_at"] = "created_at"
    tags: list[str] = []
```

---

10. Body - Multiple Parameters

Mixing Path, Query, and Body

```python
@app.put("/items/{item_id}")
```

```python
async def update_item(
    item_id: Annotated[int, Path(ge=0, le=1000)],
    q: str | None = None,
    item: Item | None = None,
):
    results = {"item_id": item_id}
    if q:
        results.update({"q": q})
    if item:
        results.update({"item": item})
    return results
```

Multiple Body Parameters

```python
@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item, user: User):
    results = {"item_id": item_id, "item": item, "user": user}
    return results
```

Singular Values in Body

```python
from fastapi import Body

@app.put("/items/{item_id}")
async def update_item(
    item_id: int, item: Item, user: User, importance: Annotated[int, Body()]
):
    results = {"item_id": item_id, "item": item, "user": user, "importance": importance}
    return results
```

Embed a Single Body Parameter

```python
@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Annotated[Item, Body(embed=True)]):
    results = {"item_id": item_id, "item": item}
    return results
```

---

11. Body - Fields

Using Field for Model Attributes

```python
from pydantic import Field

class Item(BaseModel):
    name: str
    description: str | None = Field(
        default=None, title="The description of the item", max_length=300
    )
    price: float = Field(gt=0, description="The price must be greater than zero")
    tax: float | None = None
```

Field Parameters

- `default`: Default value
- `title`: Title for documentation
- `description`: Description for documentation
- `max_length`: Maximum length for strings
- `min_length`: Minimum length for strings
- `gt`: Greater than
- `ge`: Greater than or equal
- `lt`: Less than
- `le`: Less than or equal
- `pattern`: Regex pattern
- `examples`: Examples for documentation

---

12. Body - Nested Models

List Fields

```python
class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: list[str] = []
```

Set Fields

```python
class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
```

```
    tax: float | None = None
    tags: set[str] = set()
```

Nested Models

```python
class Image(BaseModel):
    url: HttpUrl
    name: str

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: set[str] = set()
    image: Image | None = None
```

Lists of Submodels

```python
class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: set[str] = set()
    images: list[Image] | None = None
```

Deeply Nested Models

```python
class Offer(BaseModel):
    name: str
    description: str | None = None
    price: float
    items: list[Item]
```

Bodies of Pure Lists

```python
@app.post("/images/multiple/")
async def create_multiple_images(images: list[Image]):
    return images
```

Bodies of Arbitrary Dicts

```python
@app.post("/index-weights/")
async def create_index_weights(weights: dict[int, float]):
    return weights
```

---

13. Declare Request Example Data

Using Pydantic Model Config

```python
class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None

    model_config = {
        "json_schema_extra": {
            "examples": [
                {
                    "name": "Foo",
                    "description": "A very nice Item",
                    "price": 35.4,
                    "tax": 3.2,
                }
            ]
        }
    }
```

Field Additional Arguments

```python
class Item(BaseModel):
    name: str = Field(examples=["Foo"])
    description: str | None = Field(default=None, examples=["A very nice Item"])
    price: float = Field(examples=[35.4])
    tax: float | None = Field(default=None, examples=[3.2])
```

Body with Examples

```python
```

```python
@app.put("/items/{item_id}")
async def update_item(
    item_id: int,
    item: Annotated[
        Item,
        Body(
            examples=[
                {
                    "name": "Foo",
                    "description": "A very nice Item",
                    "price": 35.4,
                    "tax": 3.2,
                }
            ],
        ),
    ],
):
    results = {"item_id": item_id, "item": item}
    return results
```

OpenAPI Specific Examples

```python
@app.put("/items/{item_id}")
async def update_item(
    *,
    item_id: int,
    item: Annotated[
        Item,
        Body(
            openapi_examples={
                "normal": {
                    "summary": "A normal example",
                    "value": {
                        "name": "Foo",
                        "description": "A very nice Item",
                        "price": 35.4,
                        "tax": 3.2,
                    },
                },
                "converted": {
                    "summary": "An example with converted data",
                    "value": {
                        "name": "Bar",
                        "price": "35.4",
                    },
                },
            }
```

```
        ),
    ],
):
    results = {"item_id": item_id, "item": item}
    return results
```

---

14. Extra Data Types

Supported Data Types

Type	Description	JSON Representation
`UUID`	Universally Unique Identifier	`str`
`datetime.datetime`	Date and time	ISO 8601 string
`datetime.date`	Date	ISO 8601 string
`datetime.time`	Time	ISO 8601 string
`datetime.timedelta`	Time difference	`float` (total seconds)
`frozenset`	Immutable set	`list` (unique items)
`bytes`	Binary data	`str` (base64)
`Decimal`	Decimal number	`float`

Example with Extra Types

```python
from uuid import UUID
from datetime import datetime, time, timedelta

@app.put("/items/{item_id}")
async def read_items(
    item_id: UUID,
    start_datetime: Annotated[datetime, Body()],
    end_datetime: Annotated[datetime, Body()],
    process_after: Annotated[timedelta, Body()],
    repeat_at: Annotated[time | None, Body()] = None,
):
    start_process = start_datetime + process_after
    duration = end_datetime - start_process
    return {
        "item_id": item_id,
        "start_datetime": start_datetime,
        "end_datetime": end_datetime,
        "process_after": process_after,
        "repeat_at": repeat_at,
        "start_process": start_process,
        "duration": duration,
    }
```

```

---

## 15. Cookie Parameters

### Declaring Cookies

```python
from fastapi import Cookie

@app.get("/items/")
async def read_items(ads_id: Annotated[str | None, Cookie()] = None):
    return {"ads_id": ads_id}
```

Cookie Parameters with Validation

```python
@app.get("/items/")
async def read_items(
    ads_id: Annotated[str | None, Cookie(min_length=5, max_length=10)] = None
):
    return {"ads_id": ads_id}
```

Multiple Cookies

```python
@app.get("/items/")
async def read_items(
    session_id: Annotated[str | None, Cookie()] = None,
    user_token: Annotated[str | None, Cookie()] = None,
):
    return {"session_id": session_id, "user_token": user_token}
```

Cookie Aliases

```python
@app.get("/items/")
async def read_items(
    session_id: Annotated[str | None, Cookie(alias="session-id")] = None
):
    return {"session_id": session_id}
```

```

16. Header Parameters

Declaring Headers

```python
from fastapi import Header

@app.get("/items/")
async def read_items(user_agent: Annotated[str | None, Header()] = None):
    return {"User-Agent": user_agent}
```

Header Parameter Conversion

FastAPI automatically converts underscores to hyphens for header names:
- `user_agent` → `user-agent`

Duplicate Headers

```python
@app.get("/items/")
async def read_items(x_token: Annotated[list[str] | None, Header()] = None):
    return {"X-Token values": x_token}
```

Header Validation

```python
@app.get("/items/")
async def read_items(
    x_token: Annotated[str | None, Header(min_length=5, max_length=20)] = None
):
    return {"X-Token": x_token}
```

Disable Header Conversion

```python
@app.get("/items/")
async def read_items(
    strange_header: Annotated[str | None, Header(convert_underscores=False)] = None
):
    return {"strange_header": strange_header}
```

---

17. Cookie Parameter Models

Using Pydantic Models for Cookies

```python
class Cookies(BaseModel):
    session_id: str
    fatebook_tracker: str | None = None
    googall_tracker: str | None = None

@app.get("/items/")
async def read_items(cookies: Annotated[Cookies, Cookie()]):
    return cookies
```

Forbid Extra Cookies

```python
class Cookies(BaseModel):
    model_config = {"extra": "forbid"}
    session_id: str
    fatebook_tracker: str | None = None
    googall_tracker: str | None = None
```

---

18. Header Parameter Models

Using Pydantic Models for Headers

```python
class Headers(BaseModel):
    x_token: str
    x_version: str | None = None

@app.get("/items/")
async def read_items(headers: Annotated[Headers, Header()]):
    return headers
```

Forbid Extra Headers

```python
class Headers(BaseModel):
    model_config = {"extra": "forbid"}
    x_token: str
    x_version: str | None = None
```

---

19. Response Model - Return Type

Using Return Type Annotation

```python
@app.post("/items/")
async def create_item(item: Item) -> Item:
    return item
```

Using response_model Parameter

```python
@app.post("/items/", response_model=Item)
async def create_item(item: Item) -> Any:
    return item
```

Response Model Filtering

```python
class UserIn(BaseModel):
    username: str
    password: str
    email: str
    full_name: str | None = None

class UserOut(BaseModel):
    username: str
    email: str
    full_name: str | None = None

@app.post("/user/", response_model=UserOut)
async def create_user(user: UserIn) -> Any:
    return user
```

Inheriting Models for Filtering

```python
class BaseUser(BaseModel):
    username: str
    email: str
    full_name: str | None = None

class UserIn(BaseUser):
    password: str
```

```python
@app.post("/user/")
async def create_user(user: UserIn) -> BaseUser:
    return user
```

Excluding Default Values

```python
@app.get("/items/{item_id}", response_model=Item, response_model_exclude_unset=True)
async def read_item(item_id: str):
    return items[item_id]
```

Include and Exclude Fields

```python
@app.get(
    "/items/{item_id}/name",
    response_model=Item,
    response_model_include={"name", "description"},
)
async def read_item_name(item_id: str):
    return items[item_id]
```

---

20. Extra Models

Multiple Models for Different States

```python
class UserBase(BaseModel):
    username: str
    email: str
    full_name: str | None = None

class UserIn(UserBase):
    password: str

class UserOut(UserBase):
    pass

class UserInDB(UserBase):
    hashed_password: str
```

Union Responses

```python
@app.get("/items/{item_id}", response_model=PlaneItem | CarItem)
async def read_item(item_id: str):
    return items[item_id]
```

List of Models

```python
@app.get("/items/", response_model=list[Item])
async def read_items():
    return items
```

Arbitrary Dict Response

```python
@app.get("/keyword-weights/", response_model=dict[str, float])
async def read_keyword_weights():
    return {"foo": 2.3, "bar": 3.4}
```

---

21. Response Status Code

Setting Status Code

```python
@app.post("/items/", status_code=201)
async def create_item(name: str):
    return {"name": name}
```

Using Status Code Constants

```python
from fastapi import status

@app.post("/items/", status_code=status.HTTP_201_CREATED)
async def create_item(name: str):
    return {"name": name}
```

HTTP Status Code Categories

- `100-199`: Information
- `200-299`: Success
- `300-399`: Redirection

- `400-499`: Client errors
- `500-599`: Server errors

---

22. Form Data

Using Form

```python
from fastapi import Form

@app.post("/login/")
async def login(username: Annotated[str, Form()], password: Annotated[str, Form()]):
    return {"username": username}
```

Form with Validation

```python
@app.post("/login/")
async def login(
    username: Annotated[str, Form(min_length=3, max_length=20)],
    password: Annotated[str, Form(min_length=8)]
):
    return {"username": username}
```

Form with Default Values

```python
@app.post("/login/")
async def login(
    username: Annotated[str, Form()] = "guest",
    password: Annotated[str, Form()] = "guest123"
):
    return {"username": username}
```

Note on Form Data

- Form fields are encoded as `application/x-www-form-urlencoded`
- Multiple Form parameters cannot be mixed with Body parameters

---

23. Form Models

Using Pydantic Models for Forms

```python
class LoginForm(BaseModel):
    username: str
    password: str
    remember_me: bool = False

@app.post("/login/")
async def login(form: Annotated[LoginForm, Form()]):
    return form
```

Forbid Extra Form Fields

```python
class LoginForm(BaseModel):
    model_config = {"extra": "forbid"}
    username: str
    password: str
    remember_me: bool = False
```

Form Models with Validation

```python
class LoginForm(BaseModel):
    username: str = Field(min_length=3, max_length=20)
    password: str = Field(min_length=8)
    remember_me: bool = False
```

---

24. Request Files

Using File

```python
from fastapi import File, UploadFile

@app.post("/files/")
async def create_file(file: Annotated[bytes, File()]):
    return {"file_size": len(file)}

@app.post("/uploadfile/")
async def create_upload_file(file: UploadFile):
    return {"filename": file.filename}
```

File with Metadata

```python
@app.post("/files/")
async def create_file(
    file: Annotated[bytes, File(description="A file read as bytes")],
    fileb: Annotated[UploadFile, File(description="A file read as UploadFile")],
):
    return {
        "file_size": len(file),
        "fileb_filename": fileb.filename,
    }
```

UploadFile Methods

- `await file.read()`: Read all content
- `await file.write(data)`: Write data
- `await file.seek(offset)`: Seek position
- `await file.close()`: Close file

Multiple Files

```python
@app.post("/files/")
async def create_files(files: Annotated[list[bytes], File()]):
    return {"file_sizes": [len(file) for file in files]}

@app.post("/uploadfiles/")
async def create_upload_files(files: list[UploadFile]):
    return {"filenames": [file.filename for file in files]}
```

File Size Limits

```python
from fastapi import File, UploadFile

@app.post("/uploadfile/")
async def create_upload_file(file: UploadFile):
    # Check file size manually
    content = await file.read()
    if len(content) > 1_000_000:  # 1MB limit
        raise HTTPException(status_code=400, detail="File too large")
    return {"filename": file.filename}
```

---

25. Request Forms and Files

Combining Form and File

```python
@app.post("/files/")
async def create_file(
    file: Annotated[bytes, File()],
    fileb: Annotated[UploadFile, File()],
    token: Annotated[str, Form()],
):
    return {
        "file_size": len(file),
        "fileb_filename": fileb.filename,
        "token": token,
    }
```

Multiple Files with Form Data

```python
@app.post("/files/")
async def create_files(
    files: Annotated[list[UploadFile], File()],
    description: Annotated[str, Form()],
):
    return {
        "filenames": [file.filename for file in files],
        "description": description,
    }
```

---

26. Handling Errors

Using HTTPException

```python
from fastapi import HTTPException

@app.get("/items/{item_id}")
async def read_item(item_id: str):
    if item_id not in items:
        raise HTTPException(status_code=404, detail="Item not found")
    return {"item": items[item_id]}
```

Custom Headers in Exceptions

```python
@app.get("/items/{item_id}")
async def read_item(item_id: str):
    if item_id not in items:
        raise HTTPException(
            status_code=404,
            detail="Item not found",
            headers={"X-Error": "There goes my error"},
        )
    return {"item": items[item_id]}
```

Custom Exception Handlers

```python
from fastapi import Request
from fastapi.responses import JSONResponse

class UnicornException(Exception):
    def __init__(self, name: str):
        self.name = name

@app.exception_handler(UnicornException)
async def unicorn_exception_handler(request: Request, exc: UnicornException):
    return JSONResponse(
        status_code=418,
        content={"message": f"Oops! {exc.name} did something."},
    )

@app.get("/unicorns/{name}")
async def read_unicorn(name: str):
    if name == "yolo":
        raise UnicornException(name=name)
    return {"unicorn_name": name}
```

Override Validation Errors

```python
from fastapi.exceptions import RequestValidationError
from fastapi.responses import PlainTextResponse

@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request, exc: RequestValidationError):
    return PlainTextResponse(str(exc), status_code=400)
```

Reusing Default Exception Handlers

```python
from fastapi.exception_handlers import http_exception_handler, request_validation_exception_handler

@app.exception_handler(StarletteHTTPException)
async def custom_http_exception_handler(request, exc):
    print(f"HTTP error: {repr(exc)}")
    return await http_exception_handler(request, exc)
```

---

27. Path Operation Configuration

Tags

```python
@app.post("/items/", tags=["items"])
async def create_item(item: Item):
    return item

@app.get("/items/", tags=["items"])
async def read_items():
    return [{"name": "Foo", "price": 42}]

@app.get("/users/", tags=["users"])
async def read_users():
    return [{"username": "johndoe"}]
```

Summary and Description

```python
@app.post(
    "/items/",
    summary="Create an item",
    description="Create an item with all the information",
)
async def create_item(item: Item):
    return item
```

Description from Docstring

```python
@app.post("/items/", summary="Create an item")
async def create_item(item: Item):
    """
    Create an item with all the information:
```

```
    - **name**: each item must have a name
    - **description**: a long description
    - **price**: required
    - **tax**: if the item doesn't have tax, you can omit this
    - **tags**: a set of unique tag strings for this item
    """
    return item
```

Response Description

```python
@app.post(
    "/items/",
    response_description="The created item",
)
async def create_item(item: Item):
    return item
```

Deprecate Path Operation

```python
@app.get("/elements/", deprecated=True)
async def read_elements():
    return [{"item_id": "Foo"}]
```

Tags with Enums

```python
from enum import Enum

class Tags(Enum):
    items = "items"
    users = "users"

@app.get("/items/", tags=[Tags.items])
async def get_items():
    return ["Portal gun", "Plumbus"]
```

Exclude from OpenAPI

```python
@app.get("/items/", include_in_schema=False)
async def read_items():
    return [{"item_id": "Foo"}]
```

```

### Custom Operation ID

```python
@app.get("/items/", operation_id="some_specific_id_you_define")
async def read_items():
    return [{"item_id": "Foo"}]
```

---

28. JSON Compatible Encoder

Using jsonable_encoder

```python
from fastapi.encoders import jsonable_encoder

@app.put("/items/{id}")
def update_item(id: str, item: Item):
    json_compatible_item_data = jsonable_encoder(item)
    fake_db[id] = json_compatible_item_data
```

Common Use Cases

- Convert Pydantic models to dicts
- Convert datetime to ISO format strings
- Convert UUID to strings
- Convert custom types to JSON-compatible types

---

29. Body - Updates

PUT for Complete Replacement

```python
@app.put("/items/{item_id}")
async def update_item(item_id: str, item: Item):
    update_item_encoded = jsonable_encoder(item)
    items[item_id] = update_item_encoded
    return update_item_encoded
```

PATCH for Partial Updates

```python

```python
@app.patch("/items/{item_id}")
async def update_item(item_id: str, item: Item):
    stored_item_data = items[item_id]
    stored_item_model = Item(**stored_item_data)
    update_data = item.model_dump(exclude_unset=True)
    updated_item = stored_item_model.model_copy(update=update_data)
    items[item_id] = jsonable_encoder(updated_item)
    return updated_item
```

Using exclude_unset for Partial Updates

```python
# Only include fields that were actually sent
update_data = item.model_dump(exclude_unset=True)
```

Using model_copy for Updates

```python
updated_item = stored_item_model.model_copy(update=update_data)
```

JSON Serializable Conversion

```python
items[item_id] = jsonable_encoder(updated_item)
```

---

30. Dependencies

Basic Dependency

```python
from fastapi import Depends

async def common_parameters(q: str | None = None, skip: int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}

@app.get("/items/")
async def read_items(commons: Annotated[dict, Depends(common_parameters)]):
    return commons

@app.get("/users/")
async def read_users(commons: Annotated[dict, Depends(common_parameters)]):
    return commons
```

Dependency with Parameters

```python
async def common_parameters(
    q: str | None = None, skip: int = 0, limit: int = 100
):
    return {"q": q, "skip": skip, "limit": limit}
```

Type Alias for Dependencies

```python
CommonsDep = Annotated[dict, Depends(common_parameters)]

@app.get("/items/")
async def read_items(commons: CommonsDep):
    return commons
```

Async vs Sync Dependencies

```python
# Async dependency
async def async_dependency():
    return await some_async_operation()

# Sync dependency
def sync_dependency():
    return some_sync_operation()
```

---

31. Classes as Dependencies

Class-Based Dependency

```python
class CommonQueryParams:
    def __init__(self, q: str | None = None, skip: int = 0, limit: int = 100):
        self.q = q
        self.skip = skip
        self.limit = limit

@app.get("/items/")
async def read_items(commons: Annotated[CommonQueryParams, Depends()]):
    response = {}
    if commons.q:
```

```
        response.update({"q": commons.q})
    items = fake_items_db[commons.skip : commons.skip + commons.limit]
    response.update({"items": items})
    return response
```

Shortcut Syntax

```python
# Instead of:
commons: Annotated[CommonQueryParams, Depends(CommonQueryParams)]

# You can write:
commons: Annotated[CommonQueryParams, Depends()]
```

---

32. Sub-dependencies

Nested Dependencies

```python
async def query_extractor(q: str | None = None):
    return q

async def query_or_cookie_extractor(
    q: Annotated[str, Depends(query_extractor)],
    last_query: Annotated[str | None, Cookie()] = None,
):
    if not q:
        return last_query
    return q

@app.get("/items/")
async def read_items(query: Annotated[str, Depends(query_or_cookie_extractor)]):
    return {"q_or_cookie": query}
```

Caching Dependencies

```python
# FastAPI caches dependency results by default
# To disable caching:
async def needy_dependency(
    fresh_value: Annotated[str, Depends(get_value, use_cache=False)]
):
    return {"fresh_value": fresh_value}
```

---

33. Dependencies in Path Operation Decorators

Adding Dependencies to Decorator

```python
async def verify_token(x_token: Annotated[str, Header()]):
    if x_token != "fake-super-secret-token":
        raise HTTPException(status_code=400, detail="X-Token header invalid")

@app.get("/items/", dependencies=[Depends(verify_token)])
async def read_items():
    return [{"item": "Portal Gun"}, {"item": "Plumbus"}]
```

Multiple Dependencies

```python
async def verify_key(x_key: Annotated[str, Header()]):
    if x_key != "fake-super-secret-key":
        raise HTTPException(status_code=400, detail="X-Key header invalid")
    return x_key

@app.get("/items/", dependencies=[Depends(verify_token), Depends(verify_key)])
async def read_items():
    return [{"item": "Portal Gun"}, {"item": "Plumbus"}]
```

---

34. Global Dependencies

Adding Dependencies to the Whole App

```python
app = FastAPI(dependencies=[Depends(verify_token), Depends(verify_key)])

@app.get("/items/")
async def read_items():
    return [{"item": "Portal Gun"}, {"item": "Plumbus"}]
```

---

35. Dependencies with Yield

Basic Yield Dependency

```python
from contextlib import asynccontextmanager

@asynccontextmanager
async def get_db():
    db = DBSession()
    try:
        yield db
    finally:
        db.close()

@app.get("/items/")
async def read_items(db: Annotated[DBSession, Depends(get_db)]):
    return db.query(Item).all()
```

Database Session with Yield

```python
def get_session():
    with Session(engine) as session:
        yield session

@app.get("/items/")
async def read_items(session: Annotated[Session, Depends(get_session)]):
    return session.query(Item).all()
```

Execution Order

1. Code before `yield` runs
2. Path operation runs
3. Response is sent
4. Code after `yield` runs

---

36. Security - First Steps

OAuth2 Password Bearer

```python
from fastapi.security import OAuth2PasswordBearer

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

@app.get("/items/")
async def read_items(token: Annotated[str, Depends(oauth2_scheme)]):
```

```
    return {"token": token}
```

How It Works

1. Looks for `Authorization` header
2. Checks for `Bearer` token
3. Returns token as `str`
4. Returns 401 if missing or invalid

---

37. Get Current User

User Model

```python
class User(BaseModel):
    username: str
    email: str | None = None
    full_name: str | None = None
    disabled: bool | None = None
```

Get Current User Dependency

```python
async def get_current_user(token: Annotated[str, Depends(oauth2_scheme)]):
    user = fake_decode_token(token)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Not authenticated",
            headers={"WWW-Authenticate": "Bearer"},
        )
    return user
```

Get Current Active User

```python
async def get_current_active_user(
    current_user: Annotated[User, Depends(get_current_user)],
):
    if current_user.disabled:
        raise HTTPException(status_code=400, detail="Inactive user")
    return current_user
```

Using the Dependencies

```python
@app.get("/users/me")
async def read_users_me(
    current_user: Annotated[User, Depends(get_current_active_user)],
):
    return current_user
```

---

38. Simple OAuth2 with Password and Bearer

Token Endpoint

```python
from fastapi.security import OAuth2PasswordRequestForm

@app.post("/token")
async def login(form_data: Annotated[OAuth2PasswordRequestForm, Depends()]):
    user_dict = fake_users_db.get(form_data.username)
    if not user_dict:
        raise HTTPException(status_code=400, detail="Incorrect username or password")
    user = UserInDB(**user_dict)
    hashed_password = fake_hash_password(form_data.password)
    if not hashed_password == user.hashed_password:
        raise HTTPException(status_code=400, detail="Incorrect username or password")
    return {"access_token": user.username, "token_type": "bearer"}
```

Complete Flow

1. User sends username/password to `/token`
2. Server validates credentials
3. Server returns access token
4. Client uses token in `Authorization: Bearer <token>` header
5. Server validates token on protected endpoints

---

39. OAuth2 with Password and JWT Tokens

JWT Setup

```python
import jwt
from datetime import datetime, timedelta, timezone
```

```
SECRET_KEY = "09d25e094faa6ca2556c818166b7a9563b93f7099f6f0f4caa6cf63b88e8d3e7"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

def create_access_token(data: dict, expires_delta: timedelta | None = None):
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.now(timezone.utc) + expires_delta
    else:
        expire = datetime.now(timezone.utc) + timedelta(minutes=15)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt
```

Password Hashing

```python
from pwdlib import PasswordHash

pwd_context = PasswordHash.recommended()

def verify_password(plain_password, hashed_password):
    return pwd_context.verify(plain_password, hashed_password)

def get_password_hash(password):
    return pwd_context.hash(password)
```

Token Endpoint with JWT

```python
@app.post("/token")
async def login_for_access_token(
    form_data: Annotated[OAuth2PasswordRequestForm, Depends()],
) -> Token:
    user = authenticate_user(fake_users_db, form_data.username, form_data.password)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )
    access_token_expires = timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = create_access_token(
        data={"sub": user.username}, expires_delta=access_token_expires
    )
    return Token(access_token=access_token, token_type="bearer")
```

Get Current User with JWT

```python
async def get_current_user(token: Annotated[str, Depends(oauth2_scheme)]):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username: str = payload.get("sub")
        if username is None:
            raise credentials_exception
        token_data = TokenData(username=username)
    except InvalidTokenError:
        raise credentials_exception
    user = get_user(fake_users_db, username=token_data.username)
    if user is None:
        raise credentials_exception
    return user
```

---

40. Middleware

Creating Middleware

```python
import time

@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    start_time = time.perf_counter()
    response = await call_next(request)
    process_time = time.perf_counter() - start_time
    response.headers["X-Process-Time"] = str(process_time)
    return response
```

Middleware Execution Order

- Middlewares are executed in the order they are added
- Request path: outermost → innermost → route
- Response path: route → innermost → outermost

---

41. CORS

Basic CORS Setup

```python
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

origins = [
    "http://localhost.tiangolo.com",
    "https://localhost.tiangolo.com",
    "http://localhost",
    "http://localhost:8080",
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

CORS Parameters

- `allow_origins`: List of allowed origins
- `allow_origin_regex`: Regex for allowed origins
- `allow_methods`: List of allowed HTTP methods
- `allow_headers`: List of allowed headers
- `allow_credentials`: Allow cookies/credentials
- `expose_headers`: Headers exposed to browser
- `max_age`: Cache duration for preflight requests

---

42. SQL (Relational) Databases

Using SQLModel

```python
from sqlmodel import Field, Session, SQLModel, create_engine

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
```

```
    age: int | None = None

engine = create_engine("sqlite:///database.db")
SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session
```

CRUD Operations

```python
@app.post("/heroes/", response_model=Hero)
def create_hero(hero: Hero, session: SessionDep):
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero

@app.get("/heroes/", response_model=list[Hero])
def read_heroes(session: SessionDep):
    return session.exec(select(Hero)).all()

@app.get("/heroes/{hero_id}", response_model=Hero)
def read_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

@app.put("/heroes/{hero_id}", response_model=Hero)
def update_hero(hero_id: int, hero: HeroUpdate, session: SessionDep):
    hero_db = session.get(Hero, hero_id)
    if not hero_db:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    hero_db.sqlmodel_update(hero_data)
    session.add(hero_db)
    session.commit()
    session.refresh(hero_db)
    return hero_db

@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int, session: SessionDep):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
```

```
    session.commit()
    return {"ok": True}
```

Session Dependency

```python
SessionDep = Annotated[Session, Depends(get_session)]
```

---

43. Bigger Applications - Multiple Files

File Structure

```
.
├── app
│   ├── __init__.py
│   ├── main.py
│   ├── dependencies.py
│   └── routers
│   │   ├── __init__.py
│   │   ├── items.py
│   │   └── users.py
│   └── internal
│       ├── __init__.py
│       └── admin.py
```

APIRouter

```python
# app/routers/users.py
from fastapi import APIRouter

router = APIRouter()

@router.get("/users/", tags=["users"])
async def read_users():
    return [{"username": "Rick"}, {"username": "Morty"}]

@router.get("/users/me", tags=["users"])
async def read_user_me():
    return {"username": "fakecurrentuser"}
```

Router with Prefix and Dependencies

```python
# app/routers/items.py
from fastapi import APIRouter, Depends
from ..dependencies import get_token_header

router = APIRouter(
    prefix="/items",
    tags=["items"],
    dependencies=[Depends(get_token_header)],
    responses={404: {"description": "Not found"}},
)

@router.get("/")
async def read_items():
    return fake_items_db
```

Main App

```python
# app/main.py
from fastapi import FastAPI
from .routers import items, users
from .internal import admin
from .dependencies import get_query_token

app = FastAPI(dependencies=[Depends(get_query_token)])

app.include_router(users.router)
app.include_router(items.router)
app.include_router(
    admin.router,
    prefix="/admin",
    tags=["admin"],
    dependencies=[Depends(get_token_header)],
    responses={418: {"description": "I'm a teapot"}},
)
```

Configuring Entrypoint

```toml
# pyproject.toml
[tool.fastapi]
entrypoint = "app.main:app"
```

---

44. Stream JSON Lines

Streaming JSON Lines

```python
from fastapi.responses import StreamingResponse

@app.get("/items/stream", response_class=StreamingResponse)
async def stream_items() -> AsyncIterable[str]:
    for item in items:
        yield json.dumps(item.model_dump()) + "\n"
```

---

45. Server-Sent Events (SSE)

Basic SSE

```python
from fastapi.sse import EventSourceResponse, ServerSentEvent

@app.get("/items/stream", response_class=EventSourceResponse)
async def stream_items() -> AsyncIterable[ServerSentEvent]:
    for i, item in enumerate(items):
        yield ServerSentEvent(data=item, event="item_update", id=str(i + 1))
```

SSE with Raw Data

```python
@app.get("/logs/stream", response_class=EventSourceResponse)
async def stream_logs() -> AsyncIterable[ServerSentEvent]:
    for log_line in logs:
        yield ServerSentEvent(raw_data=log_line)
```

Resuming with Last-Event-ID

```python
@app.get("/items/stream", response_class=EventSourceResponse)
async def stream_items(
    last_event_id: Annotated[int | None, Header()] = None,
) -> AsyncIterable[ServerSentEvent]:
    start = last_event_id + 1 if last_event_id is not None else 0
    for i, item in enumerate(items):
        if i < start:
            continue
```

```
        yield ServerSentEvent(data=item, id=str(i))
```

SSE with POST

```python
@app.post("/chat/stream", response_class=EventSourceResponse)
async def stream_chat(prompt: Prompt) -> AsyncIterable[ServerSentEvent]:
    for word in words:
        yield ServerSentEvent(data=word, event="token")
    yield ServerSentEvent(raw_data="[DONE]", event="done")
```

---

46. Background Tasks

Basic Background Task

```python
from fastapi import BackgroundTasks

def write_notification(email: str, message=""):
    with open("log.txt", mode="w") as email_file:
        content = f"notification for {email}: {message}"
        email_file.write(content)

@app.post("/send-notification/{email}")
async def send_notification(email: str, background_tasks: BackgroundTasks):
    background_tasks.add_task(write_notification, email, message="some notification")
    return {"message": "Notification sent in the background"}
```

Background Tasks with Dependencies

```python
def get_query(background_tasks: BackgroundTasks, q: str | None = None):
    if q:
        background_tasks.add_task(write_log, f"found query: {q}\n")
    return q

@app.post("/send-notification/{email}")
async def send_notification(
    email: str, background_tasks: BackgroundTasks, q: Annotated[str, Depends(get_query)]
):
    background_tasks.add_task(write_log, f"message to {email}\n")
    return {"message": "Message sent"}
```

Caveats

- Background tasks run after response is sent
- For heavy computation, consider using Celery
- Background tasks share memory with the app process

---

47. Metadata and Docs URLs

App Metadata

```python
app = FastAPI(
    title="ChimichangApp",
    description="ChimichangApp API helps you do awesome stuff.",
    summary="Deadpool's favorite app.",
    version="0.0.1",
    terms_of_service="http://example.com/terms/",
    contact={
        "name": "Deadpoolio the Amazing",
        "url": "http://x-force.example.com/contact/",
        "email": "dp@x-force.example.com",
    },
    license_info={
        "name": "Apache 2.0",
        "url": "https://www.apache.org/licenses/LICENSE-2.0.html",
    },
)
```

License Identifier

```python
license_info={
    "name": "Apache 2.0",
    "identifier": "Apache-2.0",
}
```

Metadata for Tags

```python
tags_metadata = [
    {
        "name": "users",
        "description": "Operations with users. The **login** logic is also here.",
    },
    {
```

```
        "name": "items",
        "description": "Manage items. So _fancy_ they have their own docs.",
        "externalDocs": {
            "description": "Items external docs",
            "url": "https://fastapi.tiangolo.com/",
        },
    },
]

app = FastAPI(openapi_tags=tags_metadata)
```

Custom Docs URLs

```python
app = FastAPI(
    docs_url="/documentation",
    redoc_url=None,
    openapi_url="/api/v1/openapi.json",
)
```

---

48. Frontend

Serving a Frontend

```python
app.frontend("/", directory="dist")
```

Client-Side Routing

```python
app.frontend("/", directory="dist", fallback="index.html")
```

Custom 404 Page

```python
app.frontend("/", directory="dist", fallback="404.html")
```

Auto Fallback

```python
# Uses 404.html if exists, otherwise index.html
app.frontend("/", directory="dist")
```

```

### Disable Fallback

```python
app.frontend("/", directory="dist", fallback=None)
```

With APIRouter

```python
router = APIRouter()
router.frontend("/", directory="dist", fallback="index.html")
app.include_router(router, prefix="/app")
```

---

49. Static Files

Serving Static Files

```python
from fastapi.staticfiles import StaticFiles

app.mount("/static", StaticFiles(directory="static"), name="static")
```

Static Files Configuration

```python
app.mount("/static", StaticFiles(directory="static", html=True), name="static")
```

---

50. Testing

Basic Test

```python
from fastapi.testclient import TestClient

client = TestClient(app)

def test_read_main():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"msg": "Hello World"}
```
```

```

### Testing with Headers

```python
def test_read_item():
    response = client.get("/items/foo", headers={"X-Token": "coneofsilence"})
    assert response.status_code == 200
    assert response.json() == {"id": "foo", "title": "Foo"}
```

Testing with JSON Body

```python
def test_create_item():
    response = client.post(
        "/items/",
        headers={"X-Token": "coneofsilence"},
        json={"id": "foobar", "title": "Foo Bar"},
    )
    assert response.status_code == 200
    assert response.json() == {"id": "foobar", "title": "Foo Bar"}
```

Testing with Form Data

```python
def test_login():
    response = client.post(
        "/token",
        data={"username": "johndoe", "password": "secret"},
        headers={"Content-Type": "application/x-www-form-urlencoded"},
    )
    assert response.status_code == 200
    assert "access_token" in response.json()
```

Async Tests

```python
import pytest
from httpx import ASGITransport, AsyncClient

@pytest.mark.anyio
async def test_root():
    async with AsyncClient(
        transport=ASGITransport(app=app), base_url="http://test"
    ) as ac:
        response = await ac.get("/")
```
```

```
    assert response.status_code == 200
    assert response.json() == {"message": "Tomato"}
```

---

51. Debugging

Running with Debugger

```python
import uvicorn

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Debugging with VS Code

1. Go to "Debug" panel
2. "Add configuration..."
3. Select "Python"
4. Run with "Python: Current File (Integrated Terminal)"

Debugging with PyCharm

1. Open "Run" menu
2. Select "Debug..."
3. Select the file to debug

End of Part 1

FASTAPI: COMPLETE REFERENCE GUIDE (Part 2 - Advanced Topics)

Table of Contents (Part 2)

52. [Stream Data](#52-stream-data)
53. [Path Operation Advanced Configuration](#53-path-operation-advanced-configuration)
54. [Additional Status Codes](#54-additional-status-codes)
55. [Return a Response Directly](#55-return-a-response-directly)
56. [Custom Response - HTML, Stream, File, others](#56-custom-response---html-stream-file-others)
57. [Additional Responses in OpenAPI](#57-additional-responses-in-openapi)
58. [Response Cookies](#58-response-cookies)
59. [Response Headers](#59-response-headers)
60. [Response - Change Status Code](#60-response---change-status-code)

61. [Advanced Dependencies](#61-advanced-dependencies)
62. [OAuth2 Scopes](#62-oauth2-scopes)
63. [HTTP Basic Auth](#63-http-basic-auth)
64. [Using the Request Directly](#64-using-the-request-directly)
65. [Using Dataclasses](#65-using-dataclasses)
66. [Advanced Middleware](#66-advanced-middleware)
67. [Sub Applications - Mounts](#67-sub-applications---mounts)
68. [Behind a Proxy](#68-behind-a-proxy)
69. [Templates](#69-templates)
70. [WebSockets](#70-websockets)
71. [Lifespan Events](#71-lifespan-events)
72. [Testing WebSockets](#72-testing-websockets)
73. [Testing Events: lifespan and startup - shutdown](#73-testing-events-lifespan-and-startup---shutdown)
74. [Testing Dependencies with Overrides](#74-testing-dependencies-with-overrides)
75. [Async Tests](#75-async-tests)
76. [Settings and Environment Variables](#76-settings-and-environment-variables)
77. [OpenAPI Callbacks](#77-openapi-callbacks)
78. [OpenAPI Webhooks](#78-openapi-webhooks)
79. [Including WSGI - Flask, Django, others](#79-including-wsgi---flask-django-others)
80. [Generating SDKs](#80-generating-sdks)
81. [Advanced Python Types](#81-advanced-python-types)
82. [JSON with Bytes as Base64](#82-json-with-bytes-as-base64)
83. [Strict Content-Type Checking](#83-strict-content-type-checking)
84. [FastAPI CLI](#84-fastapi-cli)
85. [Editor Support](#85-editor-support)
86. [Deployment](#86-deployment)

---

52. Stream Data

Use Cases for Streaming

- Streaming AI/LLM outputs token by token
- Streaming large binary files
- Streaming video or audio
- Real-time data feeds

Basic StreamingResponse with yield

```python
from fastapi.responses import StreamingResponse

message = """
Rick: (stumbles in drunkenly, and turns on the lights) Morty! You gotta come on.
Morty: (rubs his eyes) What, Rick? What's going on?
"""
```

```
@app.get("/story/stream", response_class=StreamingResponse)
async def stream_story() -> AsyncIterable[str]:
    for line in message.splitlines():
        yield line
```

Non-async Streaming

```python
@app.get("/story/stream-no-async", response_class=StreamingResponse)
def stream_story_no_async() -> Iterable[str]:
    for line in message.splitlines():
        yield line
```

Streaming Bytes

```python
@app.get("/story/stream-bytes", response_class=StreamingResponse)
async def stream_story_bytes() -> AsyncIterable[bytes]:
    for line in message.splitlines():
        yield line.encode("utf-8")
```

Custom PNG Streaming Response

```python
class PNGStreamingResponse(StreamingResponse):
    media_type = "image/png"

@app.get("/image/stream", response_class=PNGStreamingResponse)
async def stream_image() -> AsyncIterable[bytes]:
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk
```

Using yield from

```python
@app.get("/image/stream-no-async-yield-from", response_class=PNGStreamingResponse)
def stream_image_no_async_yield_from() -> Iterable[bytes]:
    with read_image() as image_file:
        yield from image_file
```

Files and Async

When reading files, use `def` instead of `async def` to avoid blocking:

```python
@app.get("/image/stream-no-async", response_class=PNGStreamingResponse)
def stream_image_no_async() -> Iterable[bytes]:
    with read_image() as image_file:
        for chunk in image_file:
            yield chunk
```

---

53. Path Operation Advanced Configuration

OpenAPI operationId

```python
@app.get("/items/", operation_id="some_specific_id_you_define")
async def read_items():
    return [{"item_id": "Foo"}]
```

Custom Generate Unique ID Function

```python
from fastapi.routing import APIRoute

def custom_generate_unique_id(route: APIRoute) -> str:
    return route.name

app = FastAPI(generate_unique_id_function=custom_generate_unique_id)

@app.get("/items/")
async def read_items():
    return [{"item_id": "Foo"}]
```

Exclude from OpenAPI

```python
@app.get("/items/", include_in_schema=False)
async def read_items():
    return [{"item_id": "Foo"}]
```

Advanced Description with Form Feed

```python
@app.post("/items/", summary="Create an item")
async def create_item(item: Item) -> Item:
```

```
    """
    Create an item with all the information:

    - **name**: each item must have a name
    - **description**: a long description
    - **price**: required
    - **tax**: if the item doesn't have tax, you can omit this
    - **tags**: a set of unique tag strings for this item
    \f
    :param item: User input.
    """
    return item
```

OpenAPI Extensions

```python
@app.get("/items/", openapi_extra={"x-aperture-labs-portal": "blue"})
async def read_items():
    return [{"item_id": "portal-gun"}]
```

Custom OpenAPI Path Operation Schema

```python
@app.post(
    "/items/",
    openapi_extra={
        "requestBody": {
            "content": {
                "application/json": {
                    "schema": {
                        "required": ["name", "price"],
                        "type": "object",
                        "properties": {
                            "name": {"type": "string"},
                            "price": {"type": "number"},
                            "description": {"type": "string"},
                        },
                    }
                }
            },
            "required": True,
        },
    },
)
async def create_item(request: Request):
    raw_body = await request.body()
    data = magic_data_reader(raw_body)
```

```
    return data
```

Custom OpenAPI Content Type

```python
import yaml

@app.post(
    "/items/",
    openapi_extra={
        "requestBody": {
            "content": {"application/x-yaml": {"schema": Item.model_json_schema()}},
            "required": True,
        },
    },
)
async def create_item(request: Request):
    raw_body = await request.body()
    try:
        data = yaml.safe_load(raw_body)
    except yaml.YAMLError:
        raise HTTPException(status_code=422, detail="Invalid YAML")
    try:
        item = Item.model_validate(data)
    except ValidationError as e:
        raise HTTPException(status_code=422, detail=e.errors(include_url=False))
    return item
```

---

54. Additional Status Codes

Returning Additional Status Codes

```python
from fastapi.responses import JSONResponse

@app.put("/items/{item_id}")
async def upsert_item(
    item_id: str,
    name: Annotated[str | None, Body()] = None,
    size: Annotated[int | None, Body()] = None,
):
    if item_id in items:
        item = items[item_id]
        item["name"] = name
        item["size"] = size
```

```
        return item
    else:
        item = {"name": name, "size": size}
        items[item_id] = item
        return JSONResponse(status_code=status.HTTP_201_CREATED, content=item)
```

Using Status Code Constants

```python
from fastapi import status

return JSONResponse(
    status_code=status.HTTP_201_CREATED,
    content=item
)
```

---

55. Return a Response Directly

Returning a Response

```python
from fastapi.responses import JSONResponse

@app.get("/legacy/")
def get_legacy_data():
    data = """<?xml version="1.0"?>
    <shampoo>
    <Header>Apply shampoo here.</Header>
    <Body>You'll have to use soap here.</Body>
    </shampoo>
    """
    return Response(content=data, media_type="application/xml")
```

Using jsonable_encoder with Response

```python
from fastapi.encoders import jsonable_encoder

@app.put("/items/{id}")
def update_item(id: str, item: Item):
    json_compatible_item_data = jsonable_encoder(item)
    return JSONResponse(content=json_compatible_item_data)
```

Returning Custom Response

```python
@app.get("/legacy/")
def get_legacy_data():
    data = """<?xml version="1.0"?>
    <shampoo>
    <Header>Apply shampoo here.</Header>
    <Body>You'll have to use soap here.</Body>
    </shampoo>
    """
    return Response(content=data, media_type="application/xml")
```

How Response Model Works

When you declare a Response Model, FastAPI uses Pydantic to serialize the data to JSON directly, without intermediate steps:

```python
@app.post("/items/")
async def create_item(item: Item) -> Item:
    return item
```

Notes on Returning Responses

- Data is not validated, converted, or documented automatically
- Can still document with Additional Responses in OpenAPI
- Performance is better with Response Model

---

56. Custom Response - HTML, Stream, File, others

HTML Response

```python
from fastapi.responses import HTMLResponse

@app.get("/items/", response_class=HTMLResponse)
async def read_items():
    return """
    <html>
        <head>
            <title>Some HTML in here</title>
        </head>
        <body>
            <h1>Look ma! HTML!</h1>
```

```
        </body>
    </html>
    """
```

Return HTMLResponse Directly

```python
@app.get("/items/")
async def read_items():
    html_content = """
    <html>
        <head>
            <title>Some HTML in here</title>
        </head>
        <body>
            <h1>Look ma! HTML!</h1>
        </body>
    </html>
    """
    return HTMLResponse(content=html_content, status_code=200)
```

PlainTextResponse

```python
from fastapi.responses import PlainTextResponse

@app.get("/", response_class=PlainTextResponse)
async def main():
    return "Hello World"
```

RedirectResponse

```python
from fastapi.responses import RedirectResponse

@app.get("/typer")
async def redirect_typer():
    return RedirectResponse("https://typer.tiangolo.com")

# Using as response_class
@app.get("/fastapi", response_class=RedirectResponse)
async def redirect_fastapi():
    return "https://fastapi.tiangolo.com"
```

StreamingResponse

```python
from fastapi.responses import StreamingResponse

async def fake_video_streamer():
    for i in range(10):
        yield b"some fake video bytes"
        await anyio.sleep(0)

@app.get("/")
async def main():
    return StreamingResponse(fake_video_streamer())
```

FileResponse

```python
from fastapi.responses import FileResponse

@app.get("/")
async def main():
    return FileResponse("large-video-file.mp4")

# Using as response_class
@app.get("/", response_class=FileResponse)
async def main():
    return "large-video-file.mp4"
```

Available Responses Summary

Response	Media Type	Use Case
`Response`	Custom	Generic response
`HTMLResponse`	text/html	HTML pages
`PlainTextResponse`	text/plain	Plain text
`JSONResponse`	application/json	JSON data (default)
`RedirectResponse`	-	Redirects
`StreamingResponse`	Custom	Streaming data
`FileResponse`	Inferred	File downloads

Custom Response Class

```python
import orjson

class CustomORJSONResponse(Response):
    media_type = "application/json"
```

```
    def render(self, content: Any) -> bytes:
        return orjson.dumps(content, option=orjson.OPT_INDENT_2)

@app.get("/", response_class=CustomORJSONResponse)
async def main():
    return {"message": "Hello World"}
```

Default Response Class

```python
app = FastAPI(default_response_class=HTMLResponse)

@app.get("/items/")
async def read_items():
    return "<h1>Items</h1><p>This is a list of items.</p>"
```

---

57. Additional Responses in OpenAPI

Additional Response with Model

```python
class Message(BaseModel):
    message: str

@app.get("/items/{item_id}", response_model=Item, responses={404: {"model": Message}})
async def read_item(item_id: str):
    if item_id == "foo":
        return {"id": "foo", "value": "there goes my hero"}
    return JSONResponse(status_code=404, content={"message": "Item not found"})
```

Additional Media Types for Main Response

```python
@app.get(
    "/items/{item_id}",
    response_model=Item,
    responses={
        200: {
            "content": {"image/png": {}},
            "description": "Return the JSON item or an image.",
        }
    },
)
async def read_item(item_id: str, img: bool | None = None):
```

```
    if img:
        return FileResponse("image.png", media_type="image/png")
    else:
        return {"id": "foo", "value": "there goes my hero"}
```

Combining Information

```python
@app.get(
    "/items/{item_id}",
    response_model=Item,
    responses={
        404: {"model": Message, "description": "The item was not found"},
        200: {
            "description": "Item requested by ID",
            "content": {
                "application/json": {
                    "example": {"id": "bar", "value": "The bar tenders"}
                }
            },
        },
    },
)
async def read_item(item_id: str):
    if item_id == "foo":
        return {"id": "foo", "value": "there goes my hero"}
    else:
        return JSONResponse(status_code=404, content={"message": "Item not found"})
```

Combine Predefined and Custom Responses

```python
responses = {
    404: {"description": "Item not found"},
    302: {"description": "The item was moved"},
    403: {"description": "Not enough privileges"},
}

@app.get(
    "/items/{item_id}",
    response_model=Item,
    responses={**responses, 200: {"content": {"image/png": {}}}},
)
async def read_item(item_id: str, img: bool | None = None):
    if img:
        return FileResponse("image.png", media_type="image/png")
    else:
```

```
    return {"id": "foo", "value": "there goes my hero"}
```

---

58. Response Cookies

Using Response Parameter

```python
@app.post("/cookie-and-object/")
def create_cookie(response: Response):
    response.set_cookie(key="fakesession", value="fake-cookie-session-value")
    return {"message": "Come to the dark side, we have cookies"}
```

Return Response Directly

```python
@app.post("/cookie/")
def create_cookie():
    content = {"message": "Come to the dark side, we have cookies"}
    response = JSONResponse(content=content)
    response.set_cookie(key="fakesession", value="fake-cookie-session-value")
    return response
```

Cookie Parameters

- `key`: Cookie name
- `value`: Cookie value
- `max_age`: Maximum age in seconds
- `expires`: Expiration datetime
- `path`: Cookie path
- `domain`: Cookie domain
- `secure`: HTTPS only
- `httponly`: JavaScript inaccessible
- `samesite`: SameSite policy

---

59. Response Headers

Using Response Parameter

```python
@app.get("/headers-and-object/")
def get_headers(response: Response):
    response.headers["X-Cat-Dog"] = "alone in the world"
```

```
    return {"message": "Hello World"}
```

Return Response Directly

```python
@app.get("/headers/")
def get_headers():
    content = {"message": "Hello World"}
    headers = {"X-Cat-Dog": "alone in the world", "Content-Language": "en-US"}
    return JSONResponse(content=content, headers=headers)
```

Custom Headers

- Use `X-` prefix for custom headers
- Need CORS `expose_headers` for browser access

---

60. Response - Change Status Code

Using Response Parameter

```python
@app.put("/get-or-create-task/{task_id}", status_code=200)
def get_or_create_task(task_id: str, response: Response):
    if task_id not in tasks:
        tasks[task_id] = "This didn't exist before"
        response.status_code = status.HTTP_201_CREATED
    return tasks[task_id]
```

---

61. Advanced Dependencies

Parameterized Dependencies

```python
class FixedContentQueryChecker:
    def __init__(self, fixed_content: str):
        self.fixed_content = fixed_content

    def __call__(self, q: str = ""):
        if q:
            return self.fixed_content in q
        return False
```

```
checker = FixedContentQueryChecker("bar")

@app.get("/query-checker/")
async def read_query_check(fixed_content_included: Annotated[bool, Depends(checker)]):
    return {"fixed_content_in_query": fixed_content_included}
```

Dependencies with Yield and Scope

```python
def get_session():
    with Session(engine) as session:
        yield session

# Default scope is "request" - cleanup after response
@app.get("/items/")
async def read_items(session: Annotated[Session, Depends(get_session)]):
    return session.query(Item).all()

# Scope "function" - cleanup before response
@app.get("/items/")
async def read_items(
    session: Annotated[Session, Depends(get_session, scope="function")]
):
    return session.query(Item).all()
```

Dependencies with Yield and HTTPException

```python
def get_user(user_id: int, session: Annotated[Session, Depends(get_session)]):
    user = session.get(User, user_id)
    if not user:
        raise HTTPException(status_code=403, detail="Not authorized")
    session.close()  # Explicit close for early release

@app.get("/generate", dependencies=[Depends(get_user)])
def generate(query: str):
    return StreamingResponse(content=generate_stream(query))
```

Background Tasks and Dependencies with Yield

```python
# Create resources in background task itself
def background_task(user_id: int):
    # Create new session inside task
    with Session(engine) as session:
        user = session.get(User, user_id)
```

```
    # Process user...

@app.post("/process/{user_id}")
async def process_user(user_id: int):
    background_tasks.add_task(background_task, user_id)
    return {"message": "Processing started"}
```

---

62. OAuth2 Scopes

Defining Scopes

```python
from fastapi.security import OAuth2PasswordBearer, SecurityScopes

oauth2_scheme = OAuth2PasswordBearer(
    tokenUrl="token",
    scopes={
        "me": "Read information about the current user.",
        "items": "Read items."
    },
)
```

Token with Scopes

```python
@app.post("/token")
async def login_for_access_token(
    form_data: Annotated[OAuth2PasswordRequestForm, Depends()],
) -> Token:
    user = authenticate_user(fake_users_db, form_data.username, form_data.password)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )
    access_token = create_access_token(
        data={"sub": user.username, "scope": " ".join(form_data.scopes)},
        expires_delta=timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES),
    )
    return Token(access_token=access_token, token_type="bearer")
```

Declaring Scopes in Path Operations

```python
@app.get("/users/me/items/")
async def read_own_items(
    current_user: Annotated[User, Security(get_current_active_user, scopes=["items"])],
):
    return [{"item_id": "Foo", "owner": current_user.username}]
```

Using SecurityScopes

```python
async def get_current_user(
    security_scopes: SecurityScopes,
    token: Annotated[str, Depends(oauth2_scheme)]
):
    if security_scopes.scopes:
        authenticate_value = f'Bearer scope="{security_scopes.scope_str}"'
    else:
        authenticate_value = "Bearer"

    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": authenticate_value},
    )

    # Decode token, validate scopes
    for scope in security_scopes.scopes:
        if scope not in token_data.scopes:
            raise HTTPException(
                status_code=status.HTTP_401_UNAUTHORIZED,
                detail="Not enough permissions",
                headers={"WWW-Authenticate": authenticate_value},
            )
    return user
```

Dependency Tree and Scopes

```python
# Path operation read_own_items has scopes ["items"]
# get_current_active_user has scopes ["me"]
# get_current_user has no scopes
# Result: security_scopes.scopes contains ["me", "items"]

async def get_current_active_user(
    current_user: Annotated[User, Security(get_current_user, scopes=["me"])],
):
    if current_user.disabled:
```

```
        raise HTTPException(status_code=400, detail="Inactive user")
    return current_user
```

Security in Decorator Dependencies

```python
@app.get("/items/", dependencies=[Security(get_current_user, scopes=["items"])])
async def read_items():
    return [{"item": "Portal Gun"}]
```

---

63. HTTP Basic Auth

Simple HTTP Basic Auth

```python
from fastapi.security import HTTPBasic, HTTPBasicCredentials

security = HTTPBasic()

@app.get("/users/me")
def read_current_user(credentials: Annotated[HTTPBasicCredentials, Depends(security)]):
    return {"username": credentials.username, "password": credentials.password}
```

Checking Credentials with Timing Attack Protection

```python
import secrets

def get_current_username(
    credentials: Annotated[HTTPBasicCredentials, Depends(security)],
):
    current_username_bytes = credentials.username.encode("utf8")
    correct_username_bytes = b"stanleyjobson"
    is_correct_username = secrets.compare_digest(
        current_username_bytes, correct_username_bytes
    )
    current_password_bytes = credentials.password.encode("utf8")
    correct_password_bytes = b"swordfish"
    is_correct_password = secrets.compare_digest(
        current_password_bytes, correct_password_bytes
    )
    if not (is_correct_username and is_correct_password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
```

```
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Basic"},
        )
    return credentials.username
```

Why secrets.compare_digest()?

- Protects against timing attacks
- Takes the same time regardless of how many characters match
- Prevents attackers from guessing credentials by measuring response time

---

64. Using the Request Directly

Accessing Request Object

```python
from fastapi import Request

@app.get("/items/{item_id}")
def read_root(item_id: str, request: Request):
    client_host = request.client.host
    return {"client_host": client_host, "item_id": item_id}
```

Request Properties

- `request.client.host`: Client IP address
- `request.headers`: Headers dictionary
- `request.cookies`: Cookies dictionary
- `request.method`: HTTP method
- `request.url`: Full URL
- `request.query_params`: Query parameters
- `request.path_params`: Path parameters

---

65. Using Dataclasses

Dataclasses as Request Body

```python
from dataclasses import dataclass

@dataclass
class Item:
    name: str
```

```
    price: float
    description: str | None = None
    tax: float | None = None

@app.post("/items/")
async def create_item(item: Item):
    return item
```

Dataclasses in response_model

```python
@dataclass
class Item:
    name: str
    price: float
    tags: list[str] = field(default_factory=list)

@app.get("/items/next", response_model=Item)
async def read_next_item():
    return {
        "name": "Island In The Moon",
        "price": 12.99,
        "tags": ["breater"],
    }
```

Nested Dataclasses

```python
from pydantic.dataclasses import dataclass

@dataclass
class Item:
    name: str
    description: str | None = None

@dataclass
class Author:
    name: str
    items: list[Item] = field(default_factory=list)

@app.post("/authors/{author_id}/items/", response_model=Author)
async def create_author_items(author_id: str, items: list[Item]):
    return {"name": author_id, "items": items}
```

Combining Dataclasses with Other Types

```python
@app.get("/authors/", response_model=list[Author])
def get_authors():
    return [
        {
            "name": "Breaters",
            "items": [
                {"name": "Island In The Moon"},
                {"name": "Holy Buddies"},
            ],
        }
    ]
```

---

66. Advanced Middleware

Adding ASGI Middlewares

```python
from unicorn import UnicornMiddleware

app.add_middleware(UnicornMiddleware, some_config="rainbow")
```

HTTPSRedirectMiddleware

```python
from fastapi.middleware.httpsredirect import HTTPSRedirectMiddleware

app.add_middleware(HTTPSRedirectMiddleware)
```

TrustedHostMiddleware

```python
from fastapi.middleware.trustedhost import TrustedHostMiddleware

app.add_middleware(
    TrustedHostMiddleware, allowed_hosts=["example.com", "*.example.com"]
)
```

GZipMiddleware

```python
from fastapi.middleware.gzip import GZipMiddleware
```

```
app.add_middleware(GZipMiddleware, minimum_size=1000, compresslevel=5)
```

Other Middlewares

- `ProxyHeadersMiddleware`: Handle proxy headers
- `MessagePack`: MessagePack serialization
- `CORSMiddleware`: CORS handling

---

67. Sub Applications - Mounts

Mounting a FastAPI Application

```python
from fastapi import FastAPI

# Main app
app = FastAPI()

@app.get("/app")
def read_main():
    return {"message": "Hello World from main app"}

# Sub app
subapi = FastAPI()

@subapi.get("/sub")
def read_sub():
    return {"message": "Hello World from sub API"}

# Mount sub app
app.mount("/subapi", subapi)
```

Check Documentation

- Main docs: `http://127.0.0.1:8000/docs`
- Sub docs: `http://127.0.0.1:8000/subapi/docs`

Technical Details: root_path

FastAPI uses ASGI `root_path` mechanism to communicate the mount path to sub-applications.

---

68. Behind a Proxy

Proxy Forwarded Headers

Enable with CLI option:

```bash
uv run fastapi dev --forwarded-allow-ips "*"
```

Proxy with Stripped Path Prefix

```python
from fastapi import FastAPI, Request

app = FastAPI()

@app.get("/app")
def read_main(request: Request):
    return {"message": "Hello World", "root_path": request.scope.get("root_path")}
```

Setting root_path via CLI

```bash
uv run fastapi dev --root-path /api/v1
```

Setting root_path in Code

```python
app = FastAPI(root_path="/api/v1")
```

Checking Current root_path

```python
request.scope.get("root_path")
```

Testing Locally with Traefik

traefik.toml:
```toml
[entryPoints]
  [entryPoints.http]
    address = ":9999"

[providers]
  [providers.file]
    filename = "routes.toml"
```

```

**routes.toml:**
```toml
[http]
  [http.middlewares]
    [http.middlewares.api-stripprefix.stripPrefix]
      prefixes = ["/api/v1"]

  [http.routers]
    [http.routers.app-http]
      entryPoints = ["http"]
      service = "app"
      rule = "PathPrefix(`/api/v1`)"
      middlewares = ["api-stripprefix"]

  [http.services]
    [http.services.app]
      [http.services.app.loadBalancer]
        [[http.services.app.loadBalancer.servers]]
          url = "http://127.0.0.1:8000"
```

Additional Servers

```python
app = FastAPI(
    servers=[
        {"url": "https://stag.example.com", "description": "Staging environment"},
        {"url": "https://prod.example.com", "description": "Production environment"},
    ],
    root_path="/api/v1",
)
```

Disable Automatic Server from root_path

```python
app = FastAPI(root_path="/api/v1", root_path_in_servers=False)
```

---

69. Templates

Install Dependencies

```bash
uv add jinja2
```
```

```

### Using Jinja2Templates

```python
from fastapi.templating import Jinja2Templates
from fastapi.responses import HTMLResponse

templates = Jinja2Templates(directory="templates")

@app.get("/items/{id}", response_class=HTMLResponse)
async def read_item(request: Request, id: str):
    return templates.TemplateResponse(
        request=request, name="item.html", context={"id": id}
    )
```

Template File (templates/item.html)

```html
<html>
<head>
    <title>Item Details</title>
    <link href="{{ url_for('static', path='/styles.css') }}" rel="stylesheet">
</head>
<body>
    <h1><a href="{{ url_for('read_item', id=id) }}">Item ID: {{ id }}</a></h1>
</body>
</html>
```

Template Context Values

- `{{ id }}`: Renders the `id` value
- `{{ url_for('read_item', id=id) }}`: Generates URL for path operation

Templates and Static Files

```python
app.mount("/static", StaticFiles(directory="static"), name="static")
```

More Details

For testing templates, check Starlette's docs on templates.

---

70. WebSockets
```

Install websockets

```bash
uv add websockets
```

Basic WebSocket

```python
from fastapi import WebSocket

html = """
<!DOCTYPE html>
<html>
    <head>
        <title>Chat</title>
    </head>
    <body>
        <h1>WebSocket Chat</h1>
        <form action="" onsubmit="sendMessage(event)">
            <input type="text" id="messageText" autocomplete="off"/>
            <button>Send</button>
        </form>
        <ul id='messages'></ul>
        <script>
            var ws = new WebSocket("ws://localhost:8000/ws");
            ws.onmessage = function(event) {
                var messages = document.getElementById('messages')
                var message = document.createElement('li')
                var content = document.createTextNode(event.data)
                message.appendChild(content)
                messages.appendChild(message)
            };
            function sendMessage(event) {
                var input = document.getElementById("messageText")
                ws.send(input.value)
                input.value = ''
                event.preventDefault()
            }
        </script>
    </body>
</html>
"""

@app.get("/")
async def get():
    return HTMLResponse(html)
```

```python
@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    while True:
        data = await websocket.receive_text()
        await websocket.send_text(f"Message text was: {data}")
```

WebSocket with Dependencies

```python
from fastapi import WebSocket, WebSocketException, status

async def get_cookie_or_token(
    websocket: WebSocket,
    session: Annotated[str | None, Cookie()] = None,
    token: Annotated[str | None, Query()] = None,
):
    if session is None and token is None:
        raise WebSocketException(code=status.WS_1008_POLICY_VIOLATION)
    return session or token

@app.websocket("/items/{item_id}/ws")
async def websocket_endpoint(
    *,
    websocket: WebSocket,
    item_id: str,
    q: int | None = None,
    cookie_or_token: Annotated[str, Depends(get_cookie_or_token)],
):
    await websocket.accept()
    while True:
        data = await websocket.receive_text()
        await websocket.send_text(
            f"Session cookie or query token value is: {cookie_or_token}"
        )
        if q is not None:
            await websocket.send_text(f"Query parameter q is: {q}")
        await websocket.send_text(f"Message text was: {data}, for item ID: {item_id}")
```

Handling Disconnections

```python
from fastapi import WebSocketDisconnect

class ConnectionManager:
    def __init__(self):
        self.active_connections: list[WebSocket] = []
```

```
    async def connect(self, websocket: WebSocket):
        await websocket.accept()
        self.active_connections.append(websocket)

    def disconnect(self, websocket: WebSocket):
        self.active_connections.remove(websocket)

    async def send_personal_message(self, message: str, websocket: WebSocket):
        await websocket.send_text(message)

    async def broadcast(self, message: str):
        for connection in self.active_connections:
            await connection.send_text(message)

manager = ConnectionManager()

@app.websocket("/ws/{client_id}")
async def websocket_endpoint(websocket: WebSocket, client_id: int):
    await manager.connect(websocket)
    try:
        while True:
            data = await websocket.receive_text()
            await manager.send_personal_message(f"You wrote: {data}", websocket)
            await manager.broadcast(f"Client #{client_id} says: {data}")
    except WebSocketDisconnect:
        manager.disconnect(websocket)
        await manager.broadcast(f"Client #{client_id} left the chat")
```

WebSocket Exception Codes

- `WS_1008_POLICY_VIOLATION`: Policy violation
- Other valid codes from the specification

---

71. Lifespan Events

Using Lifespan Parameter

```python
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup: Load ML model
    ml_models["answer_to_everything"] = fake_answer_to_everything_ml_model
    yield
```

```
    # Shutdown: Clean up
    ml_models.clear()

app = FastAPI(lifespan=lifespan)

@app.get("/predict")
async def predict(x: float):
    result = ml_models["answer_to_everything"](x)
    return {"result": result}
```

Startup Event (Deprecated)

```python
@app.on_event("startup")
async def startup_event():
    items["foo"] = {"name": "Fighters"}
    items["bar"] = {"name": "Tenders"}
```

Shutdown Event (Deprecated)

```python
@app.on_event("shutdown")
def shutdown_event():
    with open("log.txt", mode="a") as log:
        log.write("Application shutdown")
```

Technical Details

- Underneath uses ASGI Lifespan Protocol
- Events: `startup` and `shutdown`
- For sub-applications, events only run for main app

---

72. Testing WebSockets

Basic WebSocket Test

```python
from fastapi.testclient import TestClient

def test_websocket():
    client = TestClient(app)
    with client.websocket_connect("/ws") as websocket:
        data = websocket.receive_json()
        assert data == {"msg": "Hello WebSocket"}
```

```

### Sending Messages in Test

```python
def test_websocket_chat():
    client = TestClient(app)
    with client.websocket_connect("/ws") as websocket:
        websocket.send_text("Hello!")
        data = websocket.receive_text()
        assert data == "Message text was: Hello!"
```

---

73. Testing Events: lifespan and startup - shutdown

Testing with Lifespan

```python
from contextlib import asynccontextmanager

items = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    items["foo"] = {"name": "Fighters"}
    items["bar"] = {"name": "Tenders"}
    yield
    items.clear()

app = FastAPI(lifespan=lifespan)

def test_read_items():
    assert items == {}  # Before lifespan

    with TestClient(app) as client:
        # Inside with block, lifespan runs
        assert items == {"foo": {"name": "Fighters"}, "bar": {"name": "Tenders"}}
        response = client.get("/items/foo")
        assert response.status_code == 200
        assert response.json() == {"name": "Fighters"}

    # After with block, lifespan cleanup runs
    assert items == {}
```

Testing with Startup/Shutdown Events (Deprecated)
```

```python
def test_read_items():
    with TestClient(app) as client:
        response = client.get("/items/foo")
        assert response.status_code == 200
        assert response.json() == {"name": "Fighters"}
```

---

74. Testing Dependencies with Overrides

Overriding Dependencies

```python
async def common_parameters(q: str | None = None, skip: int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}

async def override_dependency(q: str | None = None):
    return {"q": q, "skip": 5, "limit": 10}

app.dependency_overrides[common_parameters] = override_dependency

def test_override_in_items():
    response = client.get("/items/")
    assert response.status_code == 200
    assert response.json() == {
        "message": "Hello Items!",
        "params": {"q": None, "skip": 5, "limit": 10},
    }
```

Reset Overrides

```python
app.dependency_overrides = {}
```

Use Case: External Service Mocking

```python
def get_settings_override():
    return Settings(admin_email="testing_admin@example.com")

app.dependency_overrides[get_settings] = get_settings_override
```

---

75. Async Tests

Setup

```bash
uv add httpx pytest pytest-asyncio
```

Async Test Example

```python
import pytest
from httpx import ASGITransport, AsyncClient

@pytest.mark.anyio
async def test_root():
    async with AsyncClient(
        transport=ASGITransport(app=app), base_url="http://test"
    ) as ac:
        response = await ac.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "Tomato"}
```

Running Async Tests

```bash
uv run pytest
```

Warning on Lifespan Events

If your app uses lifespan, use `LifespanManager`:

```bash
uv add asgi-lifespan
```

```python
from asgi_lifespan import LifespanManager

@pytest.mark.anyio
async def test_with_lifespan():
    async with LifespanManager(app):
        async with AsyncClient(
            transport=ASGITransport(app=app), base_url="http://test"
        ) as ac:
            response = await ac.get("/")
            assert response.status_code == 200
```

```

---

## 76. Settings and Environment Variables

### Install pydantic-settings

```bash
uv add pydantic-settings
```

Basic Settings

```python
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    app_name: str = "Awesome API"
    admin_email: str
    items_per_user: int = 50

settings = Settings()

@app.get("/info")
async def info():
    return {
        "app_name": settings.app_name,
        "admin_email": settings.admin_email,
        "items_per_user": settings.items_per_user,
    }
```

Settings in Another Module

```python
# config.py
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    app_name: str = "Awesome API"
    admin_email: str
    items_per_user: int = 50

settings = Settings()

# main.py
from .config import settings
```
```

Settings in a Dependency

```python
# config.py
class Settings(BaseSettings):
    app_name: str = "Awesome API"
    admin_email: str
    items_per_user: int = 50

# main.py
from functools import lru_cache

@lru_cache
def get_settings():
    return Settings()

@app.get("/info")
async def info(settings: Annotated[Settings, Depends(get_settings)]):
    return {
        "app_name": settings.app_name,
        "admin_email": settings.admin_email,
        "items_per_user": settings.items_per_user,
    }
```

Reading .env File

Add python-dotenv:
```bash
uv add python-dotenv
```

.env file:
```env
ADMIN_EMAIL="deadpool@example.com"
APP_NAME="ChimichangApp"
```

config.py:
```python
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    app_name: str = "Awesome API"
    admin_email: str
    items_per_user: int = 50

    model_config = SettingsConfigDict(env_file=".env")
```

```

### Using lru_cache

```python
@lru_cache
def get_settings():
    return Settings()
```

- Settings created only once
- Subsequent calls return cached value
- Easy to override for testing

---

77. OpenAPI Callbacks

Creating Callback Router

```python
from fastapi import APIRouter

class Invoice(BaseModel):
    id: str
    title: str | None = None
    customer: str
    total: float

class InvoiceEvent(BaseModel):
    description: str
    paid: bool

class InvoiceEventReceived(BaseModel):
    ok: bool

invoices_callback_router = APIRouter()

@invoices_callback_router.post(
    "{$callback_url}/invoices/{$request.body.id}",
    response_model=InvoiceEventReceived
)
def invoice_notification(body: InvoiceEvent):
    pass
```

Using Callbacks in Path Operation

```python

```python
@app.post("/invoices/", callbacks=invoices_callback_router.routes)
def create_invoice(invoice: Invoice, callback_url: HttpUrl | None = None):
    """
    Create an invoice.

    This will (let's imagine) let the API user (some external developer) create an
    invoice.

    And this path operation will:

    * Send the invoice to the client.
    * Collect the money from the client.
    * Send a notification back to the API user (the external developer), as a callback.
    """
    return {"msg": "Invoice received"}
```

Callback Path Expressions

- `{$callback_url}`: Value from query parameter
- `{$request.body.id}`: Value from request body

---

78. OpenAPI Webhooks

Defining Webhooks

```python
from datetime import datetime

class Subscription(BaseModel):
    username: str
    monthly_fee: float
    start_date: datetime

@app.webhooks.post("new-subscription")
def new_subscription(body: Subscription):
    """
    When a new user subscribes to your service we'll send you a POST request
    with this data to the URL that you register for the event `new-subscription`
    in the dashboard.
    """

@app.get("/users/")
def read_users():
    return ["Rick", "Morty"]
```

Webhooks in OpenAPI

Webhooks appear in the OpenAPI schema and docs UI separately from path operations.

Requirements

- OpenAPI 3.1.0+
- FastAPI 0.99.0+

---

79. Including WSGI - Flask, Django, others

Install a2wsgi

```bash
uv add a2wsgi
```

Using WSGIMiddleware

```python
from a2wsgi import WSGIMiddleware
from flask import Flask, request
from markupsafe import escape

flask_app = Flask(__name__)

@flask_app.route("/")
def flask_main():
    name = request.args.get("name", "World")
    return f"Hello, {escape(name)} from Flask!"

app = FastAPI()

@app.get("/v2")
def read_main():
    return {"message": "Hello World"}

app.mount("/v1", WSGIMiddleware(flask_app))
```

Access

- `/v1/`: Flask application
- `/v2/`: FastAPI endpoint

Note

- Previously used `fastapi.middleware.wsgi.WSGIMiddleware`
- Now deprecated, use `a2wsgi.WSGIMiddleware`

---

80. Generating SDKs

FastAPI App with Models

```python
class Item(BaseModel):
    name: str
    price: float

class ResponseMessage(BaseModel):
    message: str

@app.post("/items/", response_model=ResponseMessage)
async def create_item(item: Item):
    return {"message": "item received"}
```

Generate TypeScript SDK with Hey API

```bash
npx @hey-api/openapi-ts -i http://localhost:8000/openapi.json -o src/client
```

Using the SDK

```typescript
import { ItemsService } from './client';

const response = await ItemsService.createItem({
    name: "Plumbus",
    price: 3
});
```

Custom Operation IDs

```python
from fastapi.routing import APIRoute

def custom_generate_unique_id(route: APIRoute):
    return f"{route.tags[0]}-{route.name}"

app = FastAPI(generate_unique_id_function=custom_generate_unique_id)
```

Preprocess OpenAPI for Better Method Names

```python
import json
from pathlib import Path

file_path = Path("./openapi.json")
openapi_content = json.loads(file_path.read_text())

for path_data in openapi_content["paths"].values():
    for operation in path_data.values():
        tag = operation["tags"][0]
        operation_id = operation["operationId"]
        to_remove = f"{tag}-"
        new_operation_id = operation_id[len(to_remove):]
        operation["operationId"] = new_operation_id

file_path.write_text(json.dumps(openapi_content))
```

---

81. Advanced Python Types

Using Union for Types

```python
from typing import Union

def say_hi(name: Union[str, None]):
    print(f"Hi {name}!")

# OR using | (preferred in Python 3.10+)
def say_hi(name: str | None):
    print(f"Hi {name}!")
```

Why Avoid Optional

```python
# 🚨 Avoid Optional
def say_hi(name: Optional[str]):  # name is still required!
    print(f"Hey {name}!")

# ✨ Use Union or |
def say_hi(name: str | None):
    print(f"Hey {name}!")
```

---

82. JSON with Bytes as Base64

Pydantic bytes for Input

```python
class DataInput(BaseModel):
    description: str
    data: bytes

    model_config = {"val_json_bytes": "base64"}

@app.post("/data")
def post_data(body: DataInput):
    content = body.data.decode("utf-8")
    return {"description": body.description, "content": content}
```

Pydantic bytes for Output

```python
class DataOutput(BaseModel):
    description: str
    data: bytes

    model_config = {"ser_json_bytes": "base64"}

@app.get("/data")
def get_data() -> DataOutput:
    data = "hello".encode("utf-8")
    return DataOutput(description="A plumbus", data=data)
```

Both Input and Output

```python
class DataInputOutput(BaseModel):
    description: str
    data: bytes

    model_config = {
        "val_json_bytes": "base64",
        "ser_json_bytes": "base64",
    }

@app.post("/data-in-out")
def post_data_in_out(body: DataInputOutput) -> DataInputOutput:
```

```
    return body
```

Base64 vs Files

- Base64 encodes binary data in strings
- Less efficient than regular files
- Use only when you must include binary data in JSON

---

83. Strict Content-Type Checking

Default Behavior

FastAPI requires a valid `Content-Type` header (`application/json`) for JSON requests.

CSRF Risk Protection

Protects against CSRF attacks in specific scenarios:
- App running locally or in internal network
- No authentication
- Browser allows requests without CORS preflight

Allowing Requests Without Content-Type

```python
app = FastAPI(strict_content_type=False)

class Item(BaseModel):
    name: str
    price: float

@app.post("/items/")
async def create_item(item: Item):
    return item
```

When to Use

- Supporting clients that don't send `Content-Type` header
- Older applications that need compatibility

---

84. FastAPI CLI

Install

```bash
uv add "fastapi[standard]"
```

Development Server

```bash
fastapi dev
```

Features:
- Auto-reload enabled
- Listens on 127.0.0.1
- Sets `FASTAPI_ENV=development`

Production Server

```bash
fastapi run
```

Features:
- Auto-reload disabled
- Listens on 0.0.0.0
- Leaves `FASTAPI_ENV` unchanged

Configure Entrypoint

```toml
# pyproject.toml
[tool.fastapi]
entrypoint = "main:app"
```

Run with Path

```bash
fastapi dev main.py
```

Run with Entrypoint

```bash
fastapi dev --entrypoint main:app
```

---

85. Editor Support

FastAPI Extension Features

- **Path Operation Explorer**: Sidebar with all route definitions
- **Route Search**: Ctrl+Shift+E (Cmd+Shift+E on macOS)
- **CodeLens Navigation**: Clickable links above test client calls
- **Deploy to FastAPI Cloud**: One-click deployment
- **Stream Application Logs**: Real-time logs from deployed app

Installation

Available for VS Code and Cursor from the Extensions panel.

Application Discovery

Auto-detects FastAPI apps by scanning for `FastAPI()` instantiations.

Configuration

```toml
# pyproject.toml
[tool.fastapi]
entrypoint = "myapp.main:app"
```

---

86. Deployment

What is Deployment?

Making your application available to users with:
- Good performance
- Stability
- High availability

Deployment Strategies

1. **Self-hosted**: Run your own server with tools like Uvicorn, Nginx
2. **Cloud Services**: Use providers that handle infrastructure
3. **FastAPI Cloud**: One-click deployment with FastAPI

Key Concepts

- **Server Workers**: Multiple processes for handling requests
- **Containers**: Docker for consistent deployment
- **HTTPS**: Secure communication
- **Load Balancing**: Distribute traffic across multiple servers

Deployment Checklist

- [ ] Set up HTTPS
- [ ] Configure CORS
- [ ] Set environment variables
- [ ] Enable logging
- [ ] Set up monitoring
- [ ] Configure database connections
- [ ] Set up backups
- [ ] Test deployment

Summary: FastAPI Complete Reference

Core Concepts
- Type hints and Pydantic models
- Path, query, and request body parameters
- Response models and status codes
- Dependencies and dependency injection
- Security and authentication

Advanced Features
- Custom responses and middlewares
- WebSockets and real-time communication
- Background tasks and lifespan events
- Testing and debugging
- Deployment and production considerations

Best Practices
- Use type hints for better editor support
- Structure big applications with APIRouter
- Use Pydantic models for data validation
- Implement proper error handling
- Write tests for your API
- Use environment variables for configuration

End of FastAPI Complete Reference Guide